CHAROTAR UNIVERSITY OF SCIENCE & TECHNOLOGY

DEVANG PATEL INSTITUTE OF ADVANCE TECHNOLOGY & RESEARCH

Department of Information Technology

Subject Name: Data Engineering

Semester: 5th

Subject Code: ITUE301

Academic year: 2026-27 [ODD]

Practical File

PRACTICAL – 2

Problem Definition

Programmatically interface with diverse file-based, API-driven, and relational source database environments. Students will write generation engines to simulate transactional workloads, profile the incoming files to automatically discover schemas, isolate data quality violations (such as null entry leaks and format divergence), and establish appropriate operational ingestion criteria.

Key Questions

1. Explain the architectural strategies used to manage severe structural schema drift when handling continuous external third-party API inputs.

Ans:

Schema drift occurs when the structure of incoming API data changes over time. To manage this problem, the following strategies are commonly used:

1. Store raw API responses before processing.
2. Detect schema changes automatically during data profiling.
3. Use flexible formats such as JSON to handle varying fields.
4. Validate incoming data before loading it into the database.
5. Log schema changes for future analysis.
6. Design ingestion pipelines that can continue processing even when optional fields change.

2. What are the structural benefits and data storage trade-offs of using semi-structured nested JSON objects over heavily normalized SQL database records?

Ans:

Nested JSON is useful for storing hierarchical and changing data structures.

Advantages:

- Easy to represent nested objects.
- Flexible schema.
- Suitable for API responses.
- Simple to exchange between applications.

Limitations:

- Difficult to perform complex SQL queries.
- More storage may be required because keys are repeated.
- Data consistency is harder to maintain.
- Relationships are not as efficient as normalized databases.

3. How do you validate data quality at the point of ingestion without creating severe processing performance bottlenecks?

Ans:

Data quality can be maintained by performing lightweight validation during ingestion.

Some common techniques are:

1. Validate only important fields.
2. Check for missing primary keys.
3. Verify email format.
4. Separate invalid records into a quarantine folder.
5. Record validation results using logs.
6. Continue processing valid records without stopping the pipeline.

Supplementary Problem

Develop a validation routine that parses a directory of messy user files, auto-quarantines records missing structural primary keys, and sends automated warning alerts to system logs.

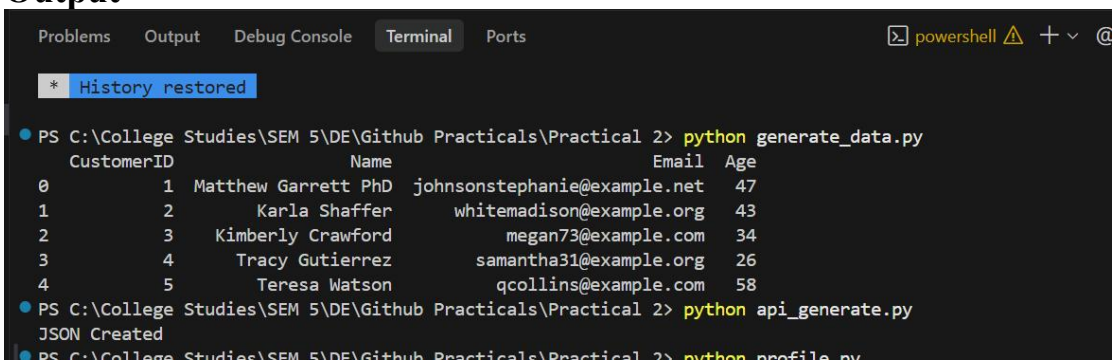
Solution

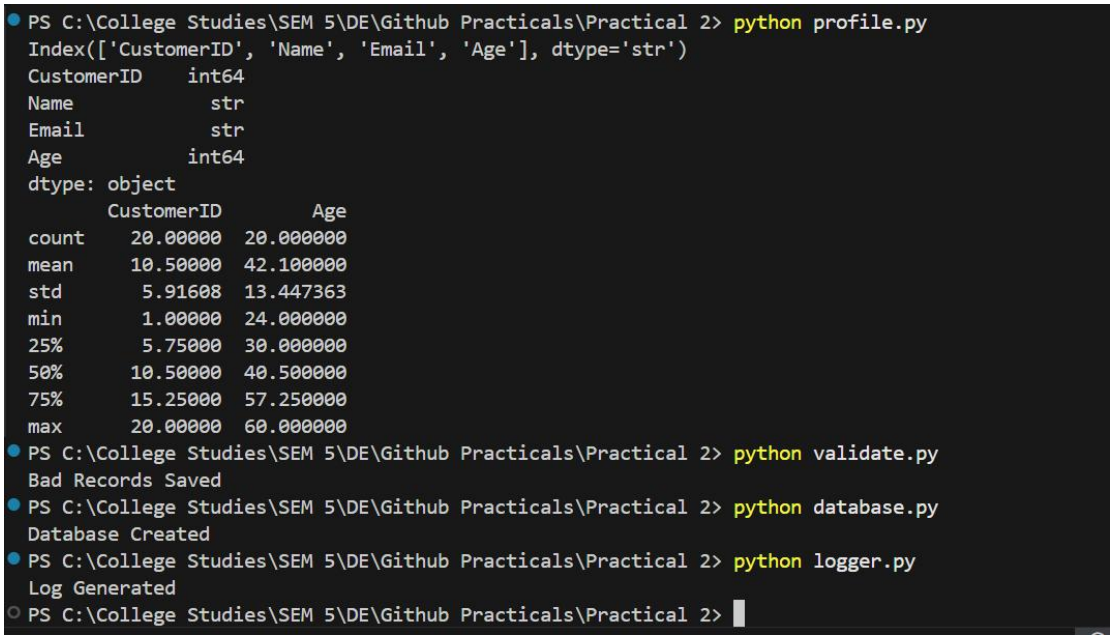
The validation program performs the following operations:

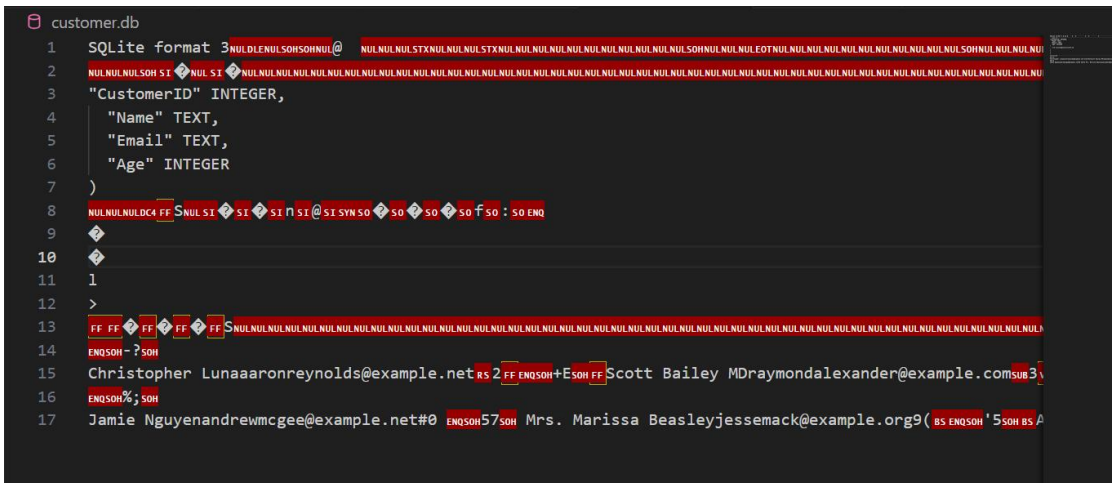
1. Reads all CSV files available inside the data folder.
2. Checks whether required columns are present.
3. Detects records with missing CustomerID.
4. Detects missing email addresses.
5. Detects invalid email formats.
6. Stores invalid records inside the quarantine folder.
7. Records validation activity in the execution.log file.
8. Continues processing remaining valid records.

This approach improves data quality while preventing invalid records from entering the database.

Output


Problems
Output
Debug Console
Terminal
Ports
powershell
* History restored
PS C:\College Studies\SEM 5\DE\Github Practicals\Practical 2> python generate_data.py
CustomerID Name Email Age
0 1 Matthew Garrett PhD johnsonstephanie@example.net 47
1 2 Karla Shaffer whitemadison@example.org 43
2 3 Kimberly Crawford megan73@example.com 34
3 4 Tracy Gutierrez samantha31@example.org 26
4 5 Teresa Watson qcollins@example.com 58
PS C:\College Studies\SEM 5\DE\Github Practicals\Practical 2> python api_generate.py
JSON Created


PS C:\College Studies\SEM 5\DE\Github Practicals\Practical 2> python profile.py
Index(['CustomerID', 'Name', 'Email', 'Age'], dtype='str')
CustomerID int64
Name str
Email str
Age int64
dtype: object
CustomerID Age
count 20.00000 20.000000
mean 10.50000 42.100000
std 5.91608 13.447363
min 1.00000 24.000000
25% 5.75000 30.000000
50% 10.50000 40.500000
75% 15.25000 57.250000
max 20.00000 60.000000
PS C:\College Studies\SEM 5\DE\Github Practicals\Practical 2> python validate.py
Bad Records Saved
PS C:\College Studies\SEM 5\DE\Github Practicals\Practical 2> python database.py
Database Created
PS C:\College Studies\SEM 5\DE\Github Practicals\Practical 2> python logger.py
Log Generated
PS C:\College Studies\SEM 5\DE\Github Practicals\Practical 2>


customer.db
SQLite format 3
"CustomerID" INTEGER,
"Name" TEXT,
"Email" TEXT,
"Age" INTEGER
)
Christopher Lunaaaronreynolds@example.net
Scott Bailey MDraymondalexander@example.com
Jamie Nguyenandrewmcgee@example.net
Mrs. Marissa Beasleyjessemack@example.org

data > { } api.json > ...


```
[
    {
        "TransactionID": 100,
        "Customer": {
            "ID": 0,
            "Name": "Customer0"
        },
        "Payment": {
            "Amount": 500,
            "Mode": "UPI"
        }
    },
    {
        "TransactionID": 101,
        "Customer": {
            "ID": 1,
            "Name": "Customer1"
        },
        "Payment": {
            "Amount": 500,
            "Mode": "UPI"
        }
    },
    {
        "TransactionID": 102,
        "Customer": {
```


Problems Output Debug Console Terminal Ports powershell

```
Database Created
PS C:\College Studies\SEM 5\DE\Github Practicals\Practical 2> python logger.py
Log Generated
PS C:\College Studies\SEM 5\DE\Github Practicals\Practical 2>
```

\

logs > execution.log

```
2026-08-03 23:32:28,348 - INFO - Validation started.
2026-08-03 23:32:28,366 - INFO - File customer.csv: 0 invalid records found.
2026-08-03 23:32:28,367 - INFO - Validation completed.
2026-08-03 23:32:39,237 - INFO - Program Started
2026-08-03 23:32:39,238 - WARNING - Missing Email Found
2026-08-03 23:32:39,238 - ERROR - Primary Key Missing
```